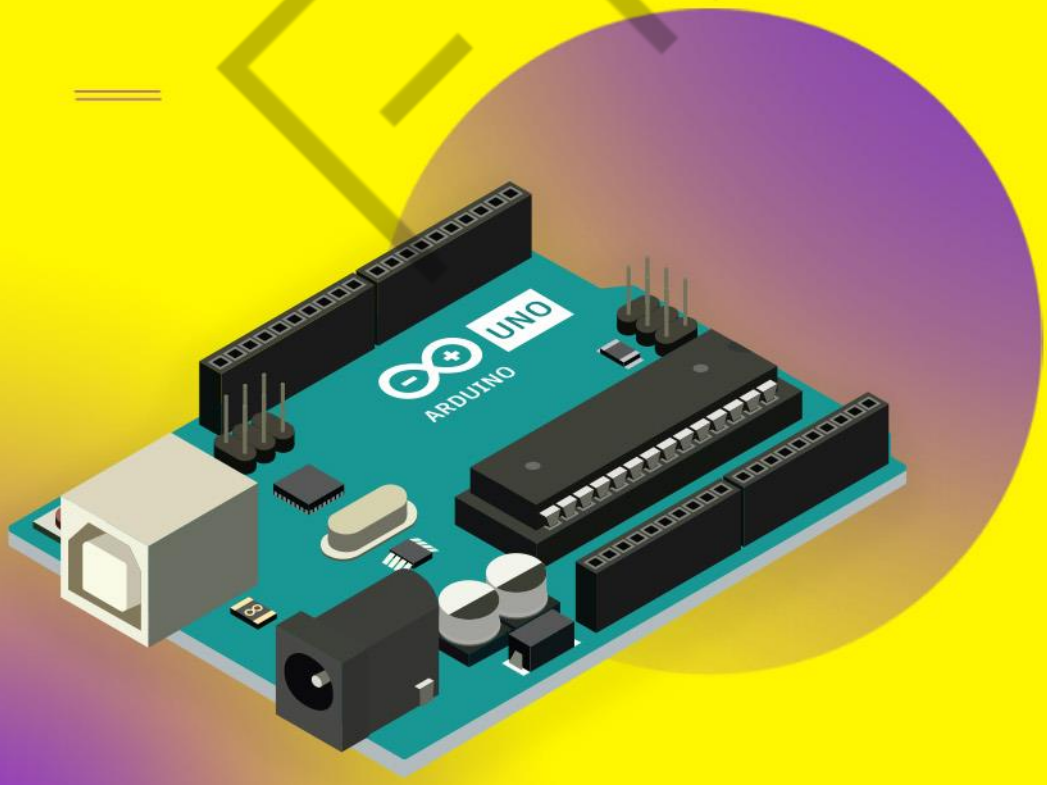
COLHEITA DE DADOS E INSIGHTS - DADOS VALIOSOS E MADUROS

CHAMANDO TODAS AS PLACAS



PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Módulo ESP8266 da Espressif 7
Figura 2 - Módulo ESP32 da Espressif 9
Figura 3 - Comparação entre ESP32 (esquerda) com ESP8266 (direita) 10
Figura 4 - Tela inicial do wokwi.com.....13
Figura 5 - Tela de inicial do Wokwi.....14
Figura 6 - Exemplo padrão usando ESP32 após carregar um template “vazio”.....14
Figura 7 - Exemplo padrão e rodando a mensagem no Monitor Serial: “Hello, ESP32!” 15
Figura 8 - Dados padrão impressos em todas as simulações referentes ao hardware 15
Figura 9 - Montagem do LED com polaridade positiva e negativa 16
Figura 10 - Circuito com LED em série com resistor de 220 Ohms. O código à esquerda será atualizado a seguir 18

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Pisca-pisca simples de um LED no pino 5 17
Código-fonte 2 - Exemplo de código testando um número par e acendendo um LED quando há sucesso 19
Código-fonte 3 - Novamente, o exemplo do pisca-pisca usando o Arduino IDE 25
Código-fonte 4 - Pisca-pisca com mensagens sendo impressas no Monitor Serial .. 26
Código-fonte 5 - Exemplo básico de lógica de programação 31
Código-fonte 6 - Sintaxe do if/else 32
Código-fonte 7 - If testando temperatura 32
Código-fonte 8 - Exemplo de uso da rotina “for” (para) 33
Código-fonte 9 - Exemplo de uso do while (enquanto) 33
Código-fonte 10 - Sintaxe do void setup() 34
Código-fonte 11 - Exemplo do que pode ser colocado no void setup() 34
Código-fonte 12 - Sintaxe do void loop() 34
Código-fonte 13 - Exemplo do que se pode codificar no void loop() 35
Código-fonte 14 - Exemplos de declarações de variáveis 36
Código-fonte 15 - Boas-práticas de como declarar variável 37
Código-fonte 16 - Exemplo de declaração de tipo inteiro 37
Código-fonte 17 - Exemplo de declaração de tipo float 38
Código-fonte 18 - Exemplo de declaração do tipo `char` com uso de aspas simples 39
Código-fonte 19 - Exemplo de declaração do tipo string com uso de aspas duplas . 39
Código-fonte 20 - Exemplo de uso de array (vetor) 40
Código-fonte 21 - Exemplo de declaração do tipo array sem se preocupar com o seu tamanho 41
Código-fonte 22 - Exemplo de array se preocupando com o seu tamanho 41
Código-fonte 23 - Exemplo de declaração do tipo `bool` 43
Código-fonte 24 - Exemplo de sintaxe usando operador && 44
Código-fonte 25 - Exemplo de sintaxe usando o operador “ou” 44
Código-fonte 26 - Exemplo de sintaxe usando a negação 44
Código-fonte 27 - Exemplo prático usando o operador E 45
Código-fonte 28 - Exemplo prático usando o operador “ou” 45
Código-fonte 29 - Exemplo prático usando a negação 45
Código-fonte 30 - Interruptor digital usando botão e LED. Quando o botão é pressionado e mantido pressionado, o LED acende e fica aceso, e quando o botão é liberado, o LED apaga 46
Código-fonte 31 - Declarando os pinos 47
Código-fonte 32 - Declarando o void setup() 47
Código-fonte 33 - Declarando o void loop() 48
Código-fonte 34 - Função que acende o LED 48
Código-fonte 35 - Função que apaga o LED 49
Código-fonte 36 - Exemplo usando dois botões que ao serem pressionados juntos, o LED acende e se mantém aceso; se soltar um botão, o LED apaga 50
Código-fonte 37 - Acende um LED se o número declarado for par 51
Código-fonte 38 - Acende um LED se 2 números forem iguais 52
Código-fonte 39 - Testando novamente números, mas agora positivos e negativos 53
Código-fonte 40 - Outro exemplo de teste de números 54

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 CHAMANDO TODAS AS PLACAS 6
1.1 Início do Arduino 6
1.2 A chegada de competidores e alternativas.....7
1.3 O impacto do ESP32.....8
1.4 Facilidade de uso para desenvolvedores 9

2 ONDE PROGRAMAR, SIMULAR E TESTAR O ESP32 13
2.1 Acessando o Wokwi.com 13
2.2 Adicionando componentes 16
2.3 Escrevendo e carregando o código.....16
2.4 Exemplo de código.....17
2.5 Simulando o código.....17
2.6 Salvando e compartilhando seu projeto 17
2.7 Praticando 18
2.8 Explorando mais recursos do Wokwi 19

3 ARDUINO IDE.....20
3.1 O Arduino IDE 21
3.2 Requisitos do sistema 21
3.3 Baixando o Arduino IDE 22
3.4 Instalando o Arduino IDE no Windows 22
3.5 No macOS.....22
3.6 No Linux 22
3.7 Primeira execução do Arduino IDE 23
3.8 Configurando a placa ESP32 23
3.9 Escrevendo e carregando seu primeiro programa: Pisca-pisca 24
3.9.1 Abra o programa 24
3.9.2 Carregar o programa.....25
3.9.3 Usando o Monitor Serial.....25
3.9.4 Salvando e organizando seus projetos 26
3.10 Documentação e comunidade.....27

4 INTRODUÇÃO À LÓGICA DE PROGRAMAÇÃO PARA ESP32 28
4.1 Python: Interpretação linha por linha.....28
4.2 C++: Compilação e Execução 29
4.3 Resumo das diferenças entre Python e C++.....30
4.4 Conceitos básicos 30

5 DECLARAÇÃO DE VARIÁVEIS.....36
5.1 int: Inteiros.....37
5.2 float: Números com ponto flutuante.....38
5.3 `char`: Caracteres 39
5.3.1 Array de `char` (C-String) 39
5.3.2 Acesso aos caracteres 40
5.3.3 std::string: String da biblioteca padrão do C++.....41
5.4 `boolean`: Valores Booleanos 43

6 OPERADORES LÓGICOS.....44
6.1 Operador AND (`&&`) 45
6.2 Operador OR (`||`) 45

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3 Operador NOT (`!`) 45
7 EXERCÍCIOS NÃO AVALIATIVOS 46
7.1 Acendendo um LED 46
7.2 Acendimento do LED usando dois botões simultâneos 49
7.3 Verificando se é par ou ímpar 50
7.4 Comparando números 51
7.5 Evoluindo o teste de números 52
7.6 Evoluindo a multiplicidade 53
REFERÊNCIAS 55

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 CHAMANDO TODAS AS PLACAS

Neste capítulo, será apresentado quatro itens importantes: a história do Arduino até chegar no ESP32; o porquê de estudar placas da família ESP em um curso de Inteligência Artificial (IA); a lógica de programação usando C/C++ para as placas Arduino/ESP; e os conceitos básicos do ESP32.

A história da evolução do Arduino e sua progressão até o ESP32 é uma narrativa interessante de inovação tecnológica e colaboração global. Acompanhe essa história.

1.1 Início do Arduino

O Arduino foi criado no início dos anos 2000 por um grupo de pesquisadores no Interaction Design Institute Ivrea, na Itália. A ideia surgiu da necessidade de criar uma plataforma de prototipagem fácil de utilizar para estudantes, “hobistas” em eletrônica e designers que não possuíam experiência em eletrônica e em programação. Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino e David Mellis foram alguns dos principais fundadores.

O primeiro protótipo do Arduino, conhecido como “Arduino Serial”, foi lançado em 2005. Ele era baseado no microcontrolador ATmega8 da Atmel e se destacou por ser acessível, open-source e simples de usar, o que facilitou sua adoção por uma ampla comunidade de desenvolvedores, estudantes e entusiastas de eletrônica.

Nos anos seguintes, o Arduino evoluiu para incluir uma série de placas com diferentes capacidades, voltadas para várias aplicações. Algumas das mais populares incluem:

- **Arduino Uno:** lançado em 2010, tornou-se a placa padrão e a mais amplamente utilizada. Era baseado no microcontrolador ATmega328.
- **Arduino Mega:** projetado para projetos mais complexos que exigem mais pinos de entrada e saída, usando o ATmega2560.
- **Arduino Nano:** uma versão compacta e mais portátil do Arduino, ideal para projetos que exigem uma placa menor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Arduino Leonardo:** introduzido em 2012, foi o primeiro Arduino a usar um microcontrolador com USB embutido (ATmega32u4), permitindo que a placa se comportasse como dispositivos Human Interface Device (HID), como teclados e mouses USB.

1.2 A chegada de competidores e alternativas

Conforme a popularidade do Arduino cresceu, outras empresas e desenvolvedores começaram a criar placas e módulos compatíveis com o Arduino, como a linha de microcontroladores PIC e ARM Cortex-M, oferecendo mais potência e funcionalidades.

Neste período, também houve um aumento na demanda por conectividade sem fio, levando ao desenvolvimento de módulos como o **ESP8266**, um módulo Wi-Fi de baixo custo e altamente popular fabricado pela multinacional chinesa Espressif Systems, que nasceu em 2008.

O **ESP8266** foi lançado em 2014 e rapidamente ganhou popularidade devido ao seu baixo custo e sua capacidade de conexão Wi-Fi integrada, algo que o Arduino tradicional não possuía sem módulos adicionais. Além disso, ele oferecia uma plataforma mais acessível e poderosa para projetos de Internet das Coisas (IoT).

Figura 1 - Módulo ESP8266 da Espressif
Fonte: Medium (2023)

Embora inicialmente fosse apenas um módulo de comunicação Wi-Fi, a comunidade descobriu que o ESP8266 tinha um processador bastante capaz e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

poderia ser programado diretamente, sem a necessidade de um microcontrolador Arduino adicional. Isso deu origem a uma nova onda de desenvolvimento IoT, tornando o ESP8266 a escolha preferida para muitos projetos.

A evolução natural do ESP8266 foi o **ESP32**, lançado pela Espressif Systems em 2016. O ESP32 é uma versão aprimorada e mais poderosa, com diversas melhorias significativas, incluindo:

- **Dual-core CPU:** com dois núcleos de 32 bits, é muito mais poderoso que o ESP8266 e a maioria dos microcontroladores Arduino.

Dica: faça uma pesquisa rápida na Internet para entender melhor o que são núcleos de 16, 32, 64 e 128 bits! Uma sugestão é o site: <https://www.tecmundo.com.br/produto/288653-diferencas-entre-processadores-32-bits-64-bits.htm>.

- **Conectividade múltipla:** além de Wi-Fi, o ESP32 inclui Bluetooth e Bluetooth Low Energy (BLE), ampliando ainda mais suas possibilidades em projetos IoT.
- **Mais GPIOs e periféricos:** com mais pinos de entrada e saída de propósitos gerais (General Purpose Input and Output – GPIOs), o ESP32 suporta mais sensores, atuadores e outros periféricos.
- **Recursos de baixo consumo de energia:** o ESP32 foi projetado com modos de economia de energia, tornando-o ideal para dispositivos alimentados por bateria.
- **Segurança:** inclui criptografia de hardware e recursos de segurança avançados para proteger a comunicação e os dados.

1.3 O impacto do ESP32

O ESP32 (ilustrado na figura a seguir) rapidamente se tornou popular em projetos de IoT e de sistemas embarcados devido à sua combinação de potência, conectividade e baixo custo. Ele é suportado por diversas plataformas de desenvolvimento, incluindo o ambiente de programação Arduino IDE (Integrated

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Development Environment), o que facilita a transição para quem já está familiarizado com o Arduino. Ainda será visto neste capítulo o que é o Arduino IDE.

O ESP32 também impulsionou o desenvolvimento de novos tipos de dispositivos conectados, como *wearables*, automação residencial, dispositivos de monitoramento remoto e muito mais.

Hoje, o ESP32 continua coexistindo com o Arduino, porém, cada um com seu nicho. Enquanto o Arduino ainda é amplamente utilizado para projetos educativos e de prototipagem rápida, o ESP32 é frequentemente escolhido para aplicações que exigem conectividade sem fio e maior capacidade de processamento.

A evolução do Arduino até o ESP32 exemplifica a rápida inovação e adaptação da comunidade de sistemas embarcados, com cada nova iteração trazendo novas possibilidades para criadores e desenvolvedores ao redor do mundo.

Figura 2 - Módulo ESP32 da Espressif
Fonte: Espressif (2020)

1.4 Facilidade de uso para desenvolvedores

Tanto o módulo ESP8266 quanto o ESP32 estão presentes em placas de desenvolvimento conhecidas, como NodeMCU, devido à necessidade de oferecer uma plataforma de desenvolvimento acessível e fácil de usar para projetos de IoT.

O termo NodeMCU refere-se a uma plataforma de código aberto, isto é, qualquer um pode construir uma placa, soldar o módulo do ESP32, adicionar alguns

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

componentes que facilitam a comunicação USB com o computador e chamar isso de NodeMCU e, também, utilizar no desenvolvimento de dispositivos de IoT ou IA.

MCU significa **Microcontroller Unit** (Unidade de Microcontrolador). Um microcontrolador é um pequeno computador em um único circuito integrado (chip) que contém um processador (CPU), memória e periféricos de entrada/saída programáveis. Os MCUs são amplamente utilizados em sistemas embarcados, onde controlam funções específicas dentro de dispositivos eletrônicos.

Portanto, o NodeMCU é um núcleo ou uma plaquinha que suporta o microcontrolador.

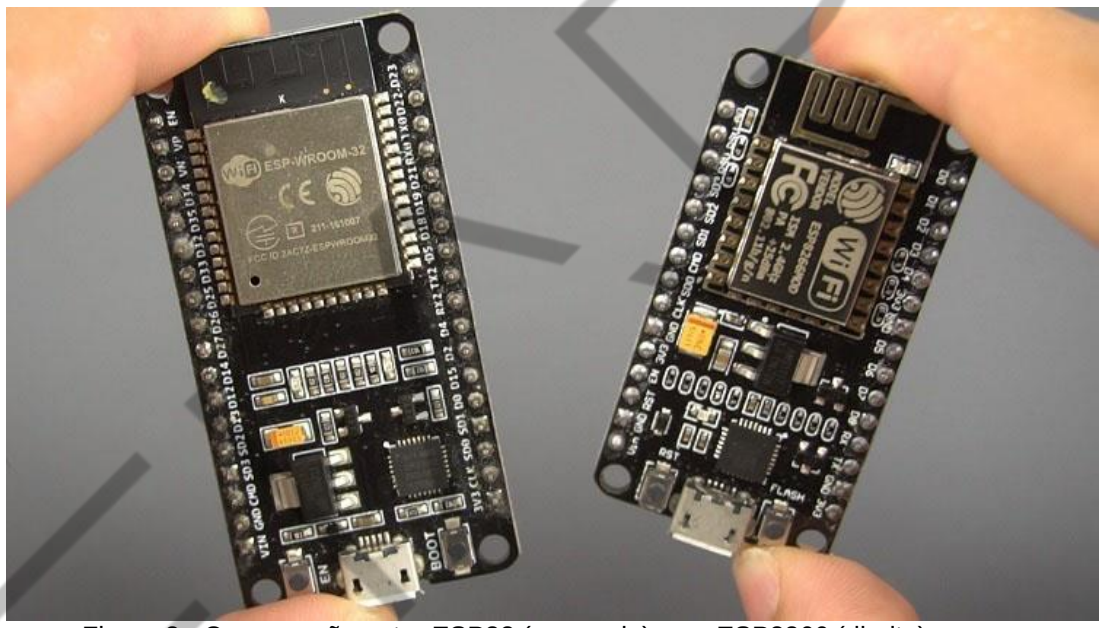


Figura 3 - Comparação entre ESP32 (esquerda) com ESP8266 (direita)
Fonte: Techeonics (2022)

O modelo mais moderno é o ESP32, porque possui Bluetooth Low Energy, além do Wi-Fi.

Estudar o ESP32 e o ESP8266 usando um NodeMCU no contexto de um curso de IA oferece diversas vantagens, especialmente quando se considera a crescente importância da Internet das Coisas e a integração de IA em dispositivos conectados. A seguir serão listadas algumas de suas vantagens:

- **Prototipagem e experimentação rápida:** o NodeMCU ESP32 e o NodeMCU ESP8266 são ideais para prototipagem rápida de dispositivos IoT, permitindo que os estudantes experimentem com a implementação de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

IA em hardware físico. Isso é crucial em cursos de inteligência artificial, onde a aplicação prática das teorias permite a compreensão dos conceitos.

Por exemplo, os estudantes podem criar protótipos de dispositivos IoT que usam algoritmos de IA para realizar tarefas específicas, como reconhecimento de voz, análise preditiva ou automação de tarefas.

- **Economia e acessibilidade:** tanto o ESP32 quanto o ESP8266 são opções de hardware de baixo custo, tornando-os acessíveis para estudantes e instituições de ensino. Isso permite que um grande número de alunos participe de projetos práticos de IA, sem a necessidade de investir em um hardware caro.

  Por exemplo, com um investimento mínimo, os estudantes podem construir dispositivos IoT equipados com IA, experimentando diretamente o uso de sensores, atuadores e algoritmos de aprendizado de máquina.

- **Aplicação prática da IA em dispositivos IoT:** o ESP32 e o ESP8266 são amplamente utilizados em projetos de IoT, onde a coleta, transmissão e processamento de dados em tempo real são essenciais. Integrar IA nesses dispositivos permite desenvolver soluções inteligentes que podem tomar decisões autônomas, otimizar processos e melhorar a eficiência de sistemas baseados em dados coletados por sensores conectados.

  Por exemplo, um dispositivo com ESP32 pode ser utilizado para coletar dados ambientais de uma fazenda como a temperatura, umidade etc., e em seguida, usar algoritmos de IA para prever padrões climáticos ou detectar anomalias.

- **Desenvolvimento de sistemas embarcados inteligentes:** o estudo do ESP32 e ESP8266 capacita os alunos a desenvolver sistemas embarcados inteligentes que podem processar dados localmente. Embora esses microcontroladores tenham recursos limitados comparados a sistemas mais poderosos, eles ainda podem executar algoritmos de IA simples, como redes neurais básicas ou algoritmos de aprendizado de máquina otimizados para dispositivos com recursos limitados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por exemplo, implementar reconhecimento facial básico, que sabe diferenciar um rosto de uma televisão usando o ESP32 e bibliotecas de IA otimizadas para dispositivos de baixo consumo.

- **Integração com plataformas de IA na nuvem:** ambos os módulos, ESP32 e ESP8266, oferecem conectividade Wi-Fi, permitindo a integração com plataformas de IA na nuvem. Os dados coletados pelos dispositivos podem ser enviados para a nuvem, onde algoritmos de IA mais complexos podem ser executados. Com isso, a nuvem pode enviar comandos de volta ao dispositivo, permitindo a criação de sistemas que combinam processamento local e remoto.

  Por exemplo, um ESP32 pode capturar imagens, enviá-las para um servidor na nuvem para análise de IA (como reconhecimento de padrões ou análise de sentimentos), e depois atuar conforme a resposta da nuvem (como enviar um alerta ou acionar um dispositivo).

- **Desenvolvimento de Habilidades Multidisciplinares:** estudar ESP32 e ESP8266 no contexto de IA ajuda a desenvolver habilidades multidisciplinares, ao combinar conhecimentos de eletrônica, programação de sistemas embarcados, redes de comunicação e algoritmos de IA. Isso é interessante no mercado de trabalho, visto que a capacidade de integrar diferentes áreas de conhecimento é cada vez mais valorizada.

  Por exemplo, alunos aprendem a programar em C++ para sistemas embarcados, a configurar redes Wi-Fi, a trabalhar com APIs de IA na nuvem e a desenvolver algoritmos que podem ser executados tanto localmente quanto remotamente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 ONDE PROGRAMAR, SIMULAR E TESTAR O ESP32

O Wokwi (wokwi.com) é uma plataforma online gratuita que permite simular projetos de eletrônica diretamente no navegador, incluindo microcontroladores como o ESP32. Com Wokwi, você pode testar seus circuitos e códigos sem precisar de um hardware físico, o que é ideal para aprender e experimentar.

2.1 Acessando o Wokwi.com

a) Abra seu navegador e acesse o site <https://wokwi.com>.

b) Crie uma conta. Embora seja possível usar o Wokwi sem uma conta, criar uma conta permite salvar seus projetos e acessá-los de qualquer lugar.

c) Na tela inicial, você verá algumas opções de placas. Escolha o ESP32, conforme ilustrado na figura a seguir.

d) Uma opção para começar um novo projeto ou explorar projetos existentes. Clique em “New Project” para iniciar um novo projeto.

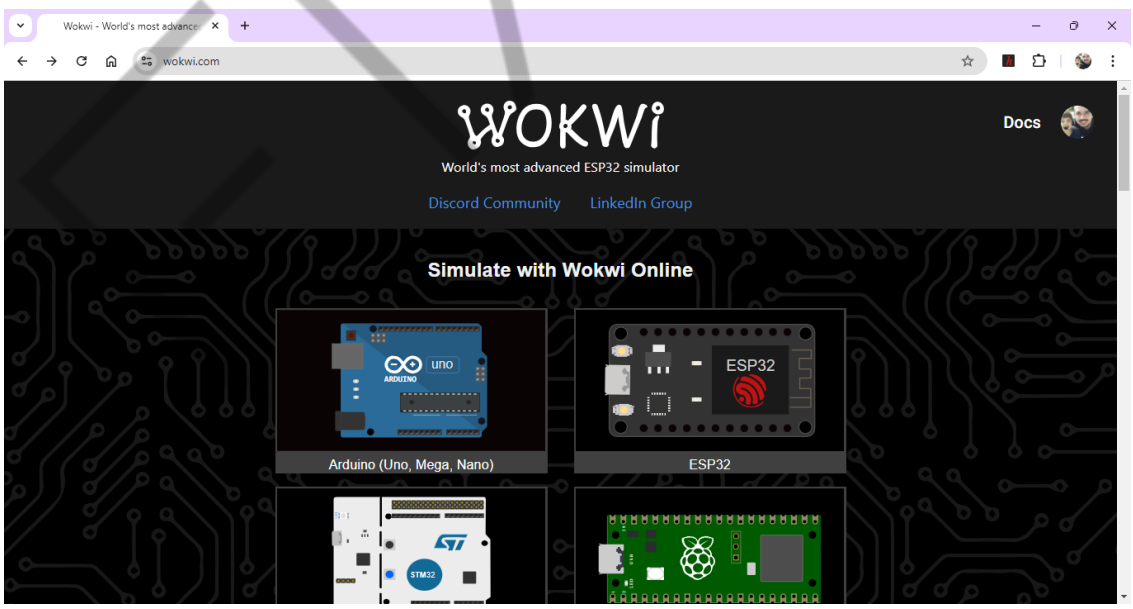


Figura 4 - Tela inicial do wokwi.com
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

e) Após clicar em ESP32, desça um pouco e vai encontrar o título Starter Templates ou Exemplos de Iniciantes. Escolha o ESP32, que é o primeiro do canto superior esquerdo.

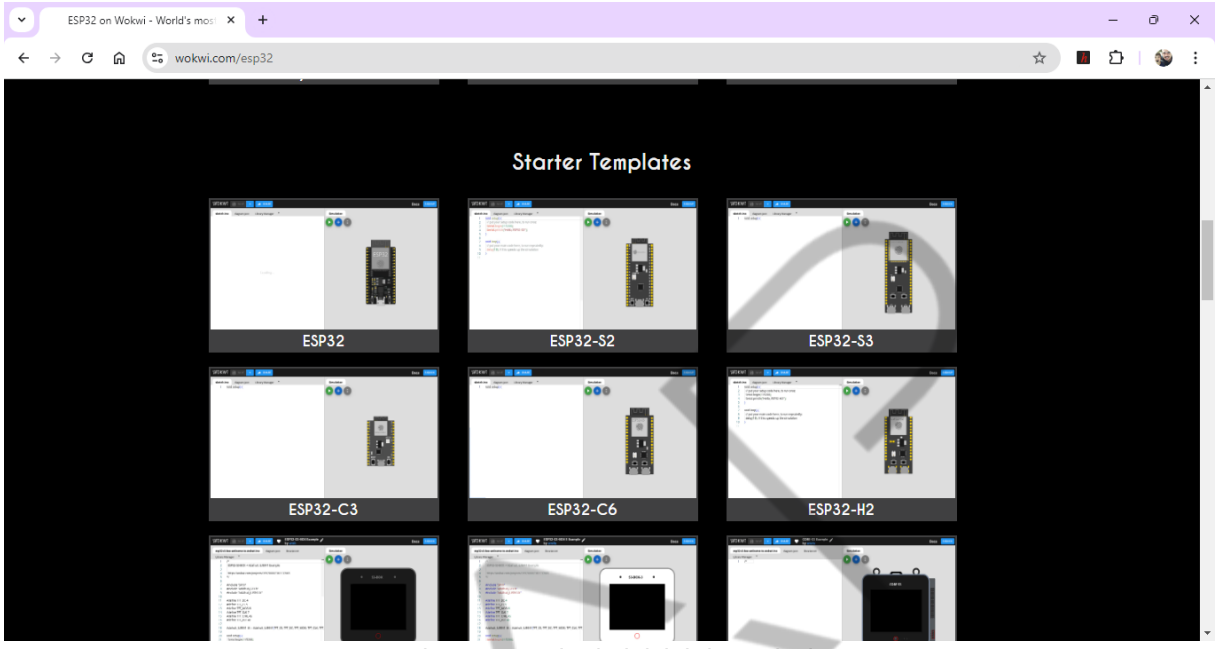


Figura 5 - Tela de inicial do Wokwi
Fonte: Elaborado pelo autor (2024)

f) Pronto! Você chegou no ambiente de desenvolvimento do ESP32. Nesse ambiente, você deve digitar o código na metade esquerda e montar o circuito eletrônico na metade direita.

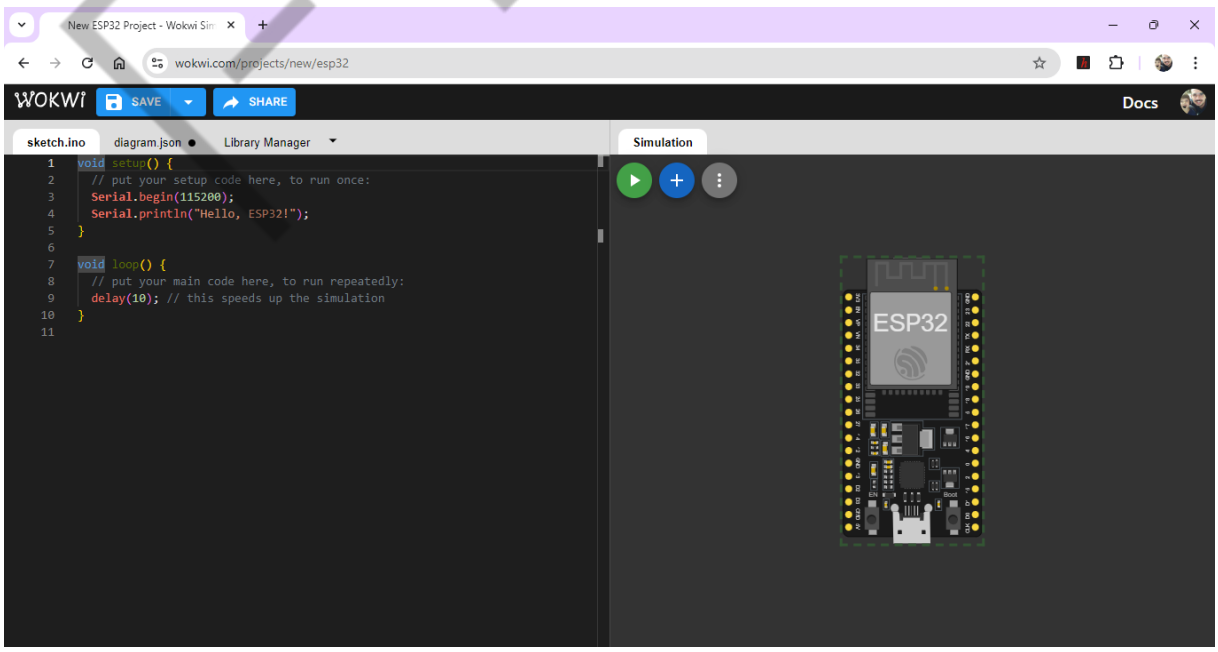


Figura 6 - Exemplo padrão usando ESP32 após carregar um template “vazio”
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

g) Para rodar o programa nativo que você acabou de abrir, clique no play verde. Vai aparecer a frase “Hello, ESP32!” no prompt da direita inferior. Esse prompt se chama Monitor Serial, pois é o local onde aparecem as mensagens advindas da função Serial.println(), que nesse exemplo inicial, está como `Serial.println("Hello, ESP32!");`

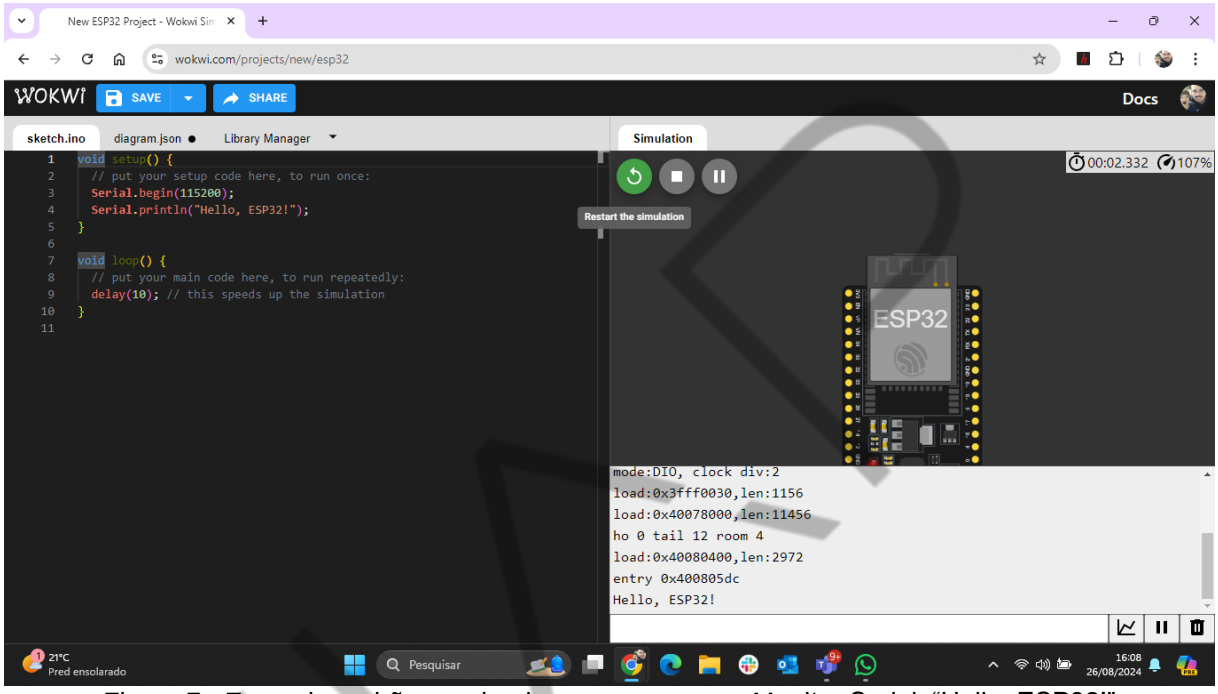


Figura 7 - Exemplo padrão e rodando a mensagem no Monitor Serial: “Hello, ESP32!”
Fonte: Elaborado pelo autor (2024)

Esse texto meio estranho que aparece antes do “Hello, ESP32!”, conforme mostra a figura a seguir, é normal e vai aparecer todas as vezes que você rodar um programa no ESP32 no Wokwi. Isso são dados de inicialização desnecessários para o nosso curso, no entanto, não tem como escondê-los.

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
```

Figura 8 - Dados padrão impressos em todas as simulações referentes ao hardware
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.2 Adicionando componentes

Quando o seu programa está no modo “pausado”, ou seja, ele não está rodando, na metade da direita, há um sinal de + azul, onde você consegue adicionar componentes eletrônicos para compor o seu projeto de IoT e IA. Você pode arrastar e soltar LEDs, botões, resistores, entre outros, para o seu circuito.

Arraste os componentes para o canvas (área de trabalho da metade direta) e conecte-os aos pinos do ESP32 clicando e arrastando para criar fios virtuais. Por exemplo, adicione um LED, conecte o pino Anodo do LED ao pino GPIO 5 do ESP32 e conecte o pino Catodo do mesmo LED ao pino GND (negativo) do ESP32.

Para descobrir qual é o pino Anodo e qual é o Catodo, basta passar o mouse sobre os pinos de LED.

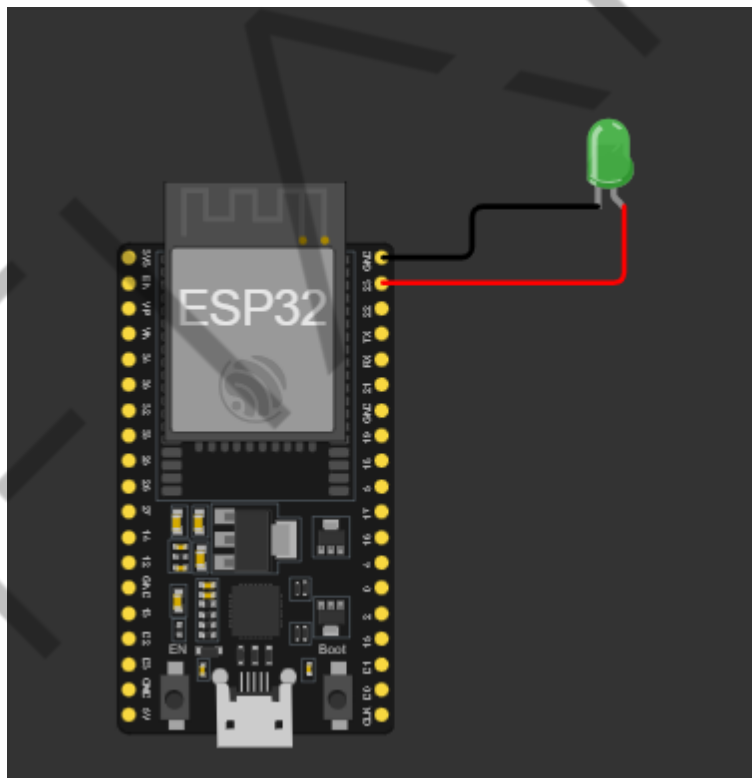


Figura 9 - Montagem do LED com polaridade positiva e negativa
Fonte: Elaborado pelo autor (2024)

2.3 Escrevendo e carregando o código

Na interface do Wokwi, você verá um editor de código na metade da esquerda. Neste local você escreve o código que será carregado no ESP32 de forma simulada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vale lembrar que não é um ESP32 real, e sim um simulador. Se você quiser comprar um, procure na internet os melhores preços. Contudo, a compra do ESP32 é totalmente opcional.

2.4 Exemplo de código

Vamos escrever um código simples para piscar um LED conectado ao pino GPIO 5 do ESP32.

```
int ledPin = 5;

void setup() {
    pinMode(ledPin, OUTPUT);
}

void loop() {
    digitalWrite(ledPin, HIGH);
    delay(1000);
    digitalWrite(ledPin, LOW);
    delay(1000);
}
```

Código-fonte 1 - Pisca-pisca simples de um LED no pino 5
Fonte: Elaborado pelo autor (2024)

Atenção: atente-se ao copiar e colar todos os códigos no Wokwi.com, remova os espaços extras em caixinhas amarelas caso apareçam.

2.5 Simulando o código

Após escrever o código, clique no botão de “Play” chamado “Start the simulation” na barra de ferramentas superior para iniciar a simulação. O LED conectado ao pino GPIO 2 deve começar a piscar, seguindo o código que você escreveu.

2.6 Salvando e compartilhando seu projeto

Se você estiver logado em sua conta, pode salvar o projeto clicando no ícone de disquete ou escolhendo “Save” no menu. Dê um nome ao seu projeto e ele será

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

salvo em sua conta. Vale lembrar que como a sua conta é gratuita, você não poderá deixar o seu projeto de modo privado.

Você pode compartilhar seu projeto com outras pessoas clicando no botão de “Share” e copiando o link gerado. Isso permite que outros vejam e simulem seu projeto diretamente no Wokwi, mas atenção, eles podem mexer ou alterar o seu código quando você compartilha via botão “Share”.

2.7 Praticando

Vamos utilizar o Wokwi para simular um dos exercícios que discutiremos ainda nesse capítulo. Neste exercício, verificaremos se um número é par ou ímpar e acenderemos um LED para indicar o resultado.

Passos:

a) Arraste um LED de qualquer cor e conecte o pino Anodo do LED ao pino GPIO 23 do ESP3.

b) Conecte um resistor de 220Ω no cátodo do LED a um resistor de 220Ω.

c) A ponta que sobrou do resistor de 220Ω você conecta no pino GND do ESP32 (que fica no canto esquerdo superior do desenho do ESP32).

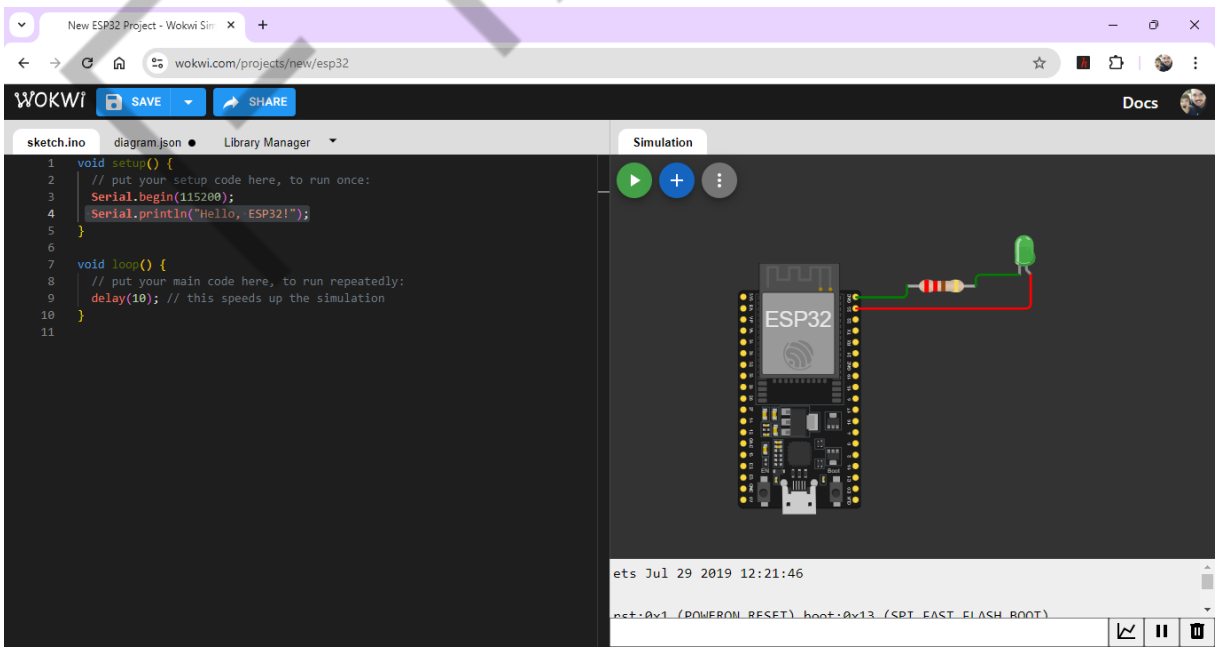


Figura 10 - Circuito com LED em série com resistor de 220 Ohms. O código à esquerda será atualizado a seguir
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

d) No editor de código (metade esquerda), escreva o seguinte código:

```
     int ledPin = 23;  // Pino GPIO 23 conectado ao LED
        int numero = 7;  // Número fornecido pelo usuário

        void setup() {
            pinMode(ledPin, OUTPUT);  // Define o pino do LED
como saída

            if (numero % 2 == 0) {
                digitalWrite(ledPin, HIGH);  // Acende o LED
se o número for par
            } else {
                digitalWrite(ledPin, LOW);   // Apaga o LED se
o número for ímpar
            }
        }

        void loop() {
            // Nada a fazer aqui, pois o estado do LED é
decidido no setup()
        }
```

Código-fonte 2 - Exemplo de código testando um número par e acendendo um LED quando há sucesso
Fonte: Elaborado pelo autor (2024)

Note que esse código só rodará uma vez por estar no void setup().

e) Clique no botão “Play” para iniciar a simulação. Observe se o LED acende ou não, dependendo do valor definido para `numero`. No exemplo do código-fonte anterior, o LED ficará apagado, pois `7` é ímpar.

2.8 Explorando mais recursos do Wokwi

Além de LEDs e botões, o Wokwi oferece uma ampla variedade de componentes como displays OLED, sensores de temperatura, motores e muito mais. Explore a biblioteca de componentes para criar projetos mais complexos. Nos próximos capítulos abordaremos com mais detalhes a eletrônica básica do ESP32.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 ARDUINO IDE

O Arduino IDE (Integrated Development Environment) é uma das plataformas mais populares para desenvolvimento de projetos em microcontroladores, especialmente para iniciantes. Sua simplicidade e a vasta comunidade de suporte tornam o Arduino IDE uma escolha interessante para quem está começando no mundo da eletrônica e da programação de dispositivos embarcados.

O IDE oferece uma interface intuitiva, com suporte para uma ampla gama de placas e componentes, além de uma biblioteca robusta que facilita a integração de sensores, atuadores e outros periféricos. Entretanto, apesar de suas inúmeras vantagens, o Arduino IDE tem algumas limitações, especialmente quando comparado a ferramentas mais modernas como o Wokwi.

Uma das principais vantagens do Arduino IDE é a sua capacidade de compilar e carregar código diretamente para o hardware físico. Isso permite que os usuários testem suas criações em tempo real, observando o comportamento dos componentes conectados à placa. Em outras palavras, se você preferir comprar uma placa ESP32 ou ESP32-CAM, poderá fazer testes interessantes.

Atenção: a FIAP não exige a compra de peças eletrônicas, ou seja, a escolha de adquirir ou não é totalmente sua. Além disso, o Arduino IDE é compatível com diversas bibliotecas e frameworks, facilitando a expansão dos projetos e a implementação de funcionalidades avançadas.

A capacidade de depurar e monitorar o funcionamento do código por meio do Monitor Serial também é um recurso poderoso, que ajuda a identificar e corrigir problemas no desenvolvimento.

Por outro lado, o Arduino IDE requer que os usuários tenham o hardware necessário em mãos para testar seus projetos. Isso pode ser uma desvantagem, especialmente para estudantes ou entusiastas que ainda não possuem todos os componentes ou que desejam experimentar antes de investir em hardware.

Além disso, a necessidade de configurar drivers e portas seriais pode ser um obstáculo para iniciantes, especialmente para quem tem iMAC, tornando o processo de configuração inicial um pouco mais complexo do que plataformas baseadas em simulação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em comparação, o Wokwi.com oferece uma solução baseada em simulação que elimina a necessidade de hardware físico. Com Wokwi, os alunos podem criar e testar seus projetos diretamente no navegador, utilizando uma interface gráfica para montar circuitos e simular o comportamento do código em diferentes microcontroladores, como o ESP32 e o Arduino.

Entretanto, o Wokwi também tem suas limitações. Embora a simulação seja precisa, ela pode não capturar todas as nuances e peculiaridades do hardware real, especialmente em projetos mais complexos que envolvem interações físicas específicas.

Enquanto o Wokwi é excelente para prototipagem e aprendizado, ele pode não ser suficiente para o desenvolvimento final de um projeto que será implementado em hardware físico. Nesse caso, a transição do ambiente de simulação para o Arduino IDE e testes com o hardware real se torna necessária, unindo as vantagens de ambas as plataformas.

3.1 O Arduino IDE

O Arduino IDE é uma plataforma de código aberto que facilita a programação e o controle de microcontroladores como o Arduino, o ESP32, o ESP8266 e outros. Com o Arduino IDE, você pode escrever código, compilar e fazer upload para o seu microcontrolador de maneira simples e intuitiva. Neste capítulo, vamos abordar o processo completo de instalação, configuração e uso do Arduino IDE, desde a configuração inicial até a escrita e execução de programas.

3.2 Requisitos do sistema

Antes de começar, certifique-se de que seu sistema atenda aos requisitos mínimos, como Windows 7 ou superior, macOS 10.10 ou superior ou qualquer distribuição moderna Linux e espaço livre em disco de pelo menos 500 MB.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3 Baixando o Arduino IDE

1. Acesse o site oficial do Arduino: <https://www.arduino.cc/en/software>.
2. Escolha sua plataforma clicando no botão de download correspondente ao seu sistema operacional (Windows, macOS, Linux).
3. Selecione “Just Download” para baixar o instalador. Não precisa pagar ou fazer donativos. Faça se quiser.

3.4 Instalando o Arduino IDE no Windows

1. Localize o arquivo baixado (geralmente chamado `Arduino-xx.xx-windows.exe`) e clique duas vezes para iniciar a instalação.
2. Caso o Windows solicite a permissão para instalar o software como administrador, clique em “Sim”.
3. Na tela de instalação, deixe todas as opções selecionadas para garantir que todos os drivers necessários sejam instalados.
4. Selecione o diretório de instalação ou deixe o padrão e clique em “Instalar”.
5. Aguarde a instalação ser concluída e clique em “Fechar”.

3.5 No macOS

1. Clique duas vezes no arquivo `.dmg` baixado.
2. Arraste o ícone do Arduino para a pasta “Aplicativos”.
3. Vá para “Aplicativos” e clique duas vezes no ícone do Arduino para iniciar.

3.6 No Linux

1. Extraia o arquivo `.tar.xz` baixado para um diretório de sua escolha.
2. No terminal, instale as dependências necessárias (geralmente `gcc`, `g++` e `libc6`).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3. No terminal, navegue até a pasta extraída e execute o script de instalação com o comando `./install.sh`.

3.7 Primeira execução do Arduino IDE

1. Após a instalação, abra o Arduino IDE. A interface principal será carregada, mostrando uma área de código em branco, menus no topo e uma barra de status na parte inferior.
2. Use um cabo USB para conectar sua placa Arduino ao computador. Atenção para esse item, pois o cabo precisa ser de dados e não somente um cabo que apenas carrega celulares. Para testar o seu cabo, espete um celular plugado no cabo em questão e veja se o sistema operacional reconhece o seu dispositivo. Se reconhecer, significa que você tem um cabo de dados em mãos.

3.8 Configurando a placa ESP32

Para programar o ESP32 utilizando o Arduino IDE, você precisará instalar o suporte a essa placa. Siga os seguintes passos:

- **Adicionar Suporte ao ESP32:**
  - **Passo 1:** no Arduino IDE, vá para “Arquivo (File)” > “Preferências (Preferences)”.
  - **Passo 2:** na janela de preferências, localize o campo chamado URL Adicional para Gerenciadores de Placas (Additional Boards Manager URLs).
  - **Passo 3:** cole a seguinte URL nesse campo: https://dl.espressif.com/dl/package_esp32_index.json
  - **Passo 4:** clique em “OK” para fechar a janela de preferências.
- **Instalar as Placas ESP32:**
  - **Passo 1:** vá para “Ferramentas (Tools)” > “Placa (Board)” > “Gerenciador de Placas (Boards Manager)”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Passo 2:** no Gerenciador de Placas, digite “ESP32” na barra de pesquisa.
- **Passo 3:** selecione a entrada “esp32 by Espressif Systems”, e clique em “Instalar”.
- **Passo 4:** aguarde a instalação ser concluída. Isso pode levar alguns minutos dependendo da sua conexão à internet.

- **Selecionar a Placa ESP32:**
  - **Passo 1:** depois de instalar o suporte ao ESP32, vá para “Ferramentas (Tools)” > “Placa (Board)” e selecione a placa ESP32 que você está utilizando. Por exemplo, se você estiver usando um “ESP32 Dev Module”, selecione-o na lista, ou o “ESP32-CAM”, selecione-o.
- **Selecionar a Porta Serial:**
  - **Passo 1:** conecte sua placa ESP32 ao computador usando um cabo USB.
  - **Passo 2:** no Arduino IDE, vá para “Ferramentas (Tools)” > “Porta (Port)” e selecione a porta COM associada ao seu ESP32. No Windows, será algo como `COM3`, `COM4` etc. No macOS/Linux, será algo como `/dev/ttyUSB0` ou `/dev/tty.SLAB_USBtoUART`.

Dica importante: se você não sabe qual porta USB está associada à sua placa, desplugue a placa da porta USB, veja a listagem de portas disponíveis em “Ferramentas (Tools)” > “Porta (Port)” sem a placa estar conectada, depois, reconecte a placa, e veja qual porta COM apareceu de novidade. Lembrando: você precisa fechar as telas “Ferramentas (Tools)” > “Porta (Port)” para que o software Arduino IDE atualize internamente a chamada das portas.

3.9 Escrevendo e carregando seu primeiro programa: Pisca-pisca

O programa “Blink” é o clássico exemplo inicial. Ele faz um LED na placa Arduino/ESP32 piscar em intervalos regulares.

3.9.1 Abra o programa

No Arduino IDE, vá para “Arquivo (File)” > “Exemplos (Examples)” > “01.Basics” > “Blink”. Isso abrirá o código do exemplo a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      // O LED embutido na placa está conectado ao pino digital
13 no Arduino Uno e pino 4 no ESP32 ou ESP32-CAM

        void setup() {
          pinMode(4, OUTPUT);           // Define o pino 4 como
saída
        }

        void loop() {
          digitalWrite(4, HIGH);   // Acende o LED
          delay(1000);                  // Aguarda 1 segundo
          digitalWrite(4, LOW);    // Apaga o LED
          delay(1000);                      //    Aguarda    1
segundo
        }
```

Código-fonte 3 - Novamente, o exemplo do pisca-pisca usando o Arduino IDE
Fonte: Elaborado pelo autor (2024)

É importante prestar atenção ao copiar e colar os códigos no Wokwi.com, removendo os espaços extras em caixinhas amarelas caso apareçam.

Observações:

- setup() é executado uma vez quando o programa começa. Configura o pino como saída.
- loop() é executado repetidamente. Alterna o LED entre ligado e desligado, com um atraso de 1 segundo.

3.9.2 Carregar o programa

Clique no botão de “Verificar” (ícone de check) para compilar o código. Em seguida, clique no botão de “Carregar” (ícone de seta), e o código será enviado para a placa Arduino. O LED na placa começará a piscar.

3.9.3 Usando o Monitor Serial

O Monitor Serial é uma ferramenta no Arduino IDE que permite a comunicação entre o computador e a placa Arduino/ESP via porta serial. Ele é útil para depuração e para interação com o Arduino em tempo real.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para abrir o Monitor Serial, vá para “Ferramentas” > “Monitor Serial” ou clique no ícone de lupa no canto superior direito.

Adicione ao código Blink para imprimir mensagens no Monitor Serial o seguinte exemplo:

```
     void setup() {
          Serial.begin(9600);      //  Inicia  a  comunicação
serial a 9600 bps
          pinMode(4, OUTPUT);      //  Define  o  pino  4  como
saída
        }

     void loop() {
          digitalWrite(4, HIGH);   // Acende o LED
          Serial.println("LED ON");     //  Envia  "LED  ON"  ao
Monitor Serial
          delay(1000);                     //    Aguarda    1
segundo

          digitalWrite(4, LOW);        // Apaga o LED
          Serial.println("LED  OFF");  //  Envia  "LED  OFF"  ao
Monitor Serial
          delay(1000);   // Aguarda 1 segundo
        }
```

Código-fonte 4 - Pisca-pisca com mensagens sendo impressas no Monitor Serial
Fonte: Elaborado pelo autor (2024)

Carregue o código na placa e abra o Monitor Serial. Você verá “LED ON” e “LED OFF” sendo exibidos alternadamente, correspondendo ao estado do LED.

3.9.4 Salvando e organizando seus projetos

No Arduino IDE, vá para “Arquivo” > “Salvar Como”. Em seguida, dê um nome ao seu projeto e selecione o local onde deseja salvá-lo.

Cada projeto deve ser salvo em uma pasta separada, com o mesmo nome do arquivo `.ino`. Isso mantém os arquivos do projeto organizados e evita problemas de compilação.

Para abrir um projeto salvo, vá para “Arquivo” > “Abrir” e navegue até a pasta do projeto. Selecione o arquivo `.ino` para carregá-lo no Arduino.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.10 Documentação e comunidade

O site oficial do Arduino (<www.arduino.cc>) é um excelente recurso para documentação, tutoriais e exemplos de código. Além disso, você pode participar de fóruns como o Arduino Forum para tirar dúvidas, compartilhar projetos e aprender com outros entusiastas.

Comece com projetos simples, como controlar LEDs, botões e motores. Isso ajudará a consolidar o conhecimento básico, e gradualmente explorar sensores, comunicação sem fio (como o ESP8266/ESP32) e integração com outras plataformas (como IoT).

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 INTRODUÇÃO À LÓGICA DE PROGRAMAÇÃO PARA ESP32

Depois de ter visto várias informações relevantes sobre o ESP32, agora vamos entrar na lógica de programação do ESP32.

Primeiro, a lógica de programação é a base de qualquer desenvolvimento de software. Ela envolve a sequência de instruções que um computador deve seguir para executar uma tarefa específica.

No contexto de sistemas embarcados como o ESP32, é essencial entender a lógica por trás dos algoritmos para controlar hardware, sensores e outros componentes de forma eficiente. Pode ficar tranquilo, pois esse conteúdo não será muito diferente do que você já viu na Fase 1 com Python.

A lógica desse capítulo é praticamente a mesma do Python, porém, vamos contextualizar no mundo do ESP32 e usar outro código de programação, o C++.

O código em C++ é diferente do código em Python no que diz respeito à forma como é executado. Vamos explorar essas diferenças.

4.1 Python: Interpretação linha por linha

A linguagem em Python é uma linguagem interpretada, isso significa que o código é executado linha por linha por um interpretador. Quando você executa um script Python, o interpretador lê e executa cada linha de código na sequência em que aparece.

Se houver um erro em alguma linha, o interpretador interrompe a execução e imediatamente exibe uma mensagem de erro. Isso facilita a depuração (correção de erros) porque você sabe exatamente onde o problema ocorreu. Você pode até executar um código em Python cujo erro só vai quebrar a funcionalidade quando de fato chegar nele.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2 C++: Compilação e Execução

O C++ é uma linguagem compilada, ou seja, o código-fonte escrito em C++ precisa passar por um processo de compilação antes de ser executado. A compilação converte o código C++ em um código de máquina binário que o computador/microcontrolador pode entender e executar diretamente. As etapas do processo de compilação são:

- **Pré-processamento:** instruções de pré-processadores como `#include` e `#define` são processadas.
- **Compilação:** o código-fonte é traduzido para código de máquina (normalmente gerando arquivos objeto).
- **Linkagem:** os arquivos objeto são combinados com bibliotecas para criar o executável final.

Depois que o código é compilado com sucesso em um executável, ele é executado como um todo, sem a necessidade de um interpretador. Isso geralmente resulta em um desempenho mais rápido, porque o código já está convertido em uma forma que o processador pode executar diretamente. É o famoso *.exe que você encontra como programa executável em seu computador e em diversos diretórios. Contudo, no mundo do Arduino/ESP32, não vamos ter o arquivo *.exe e sim, um arquivo *.ino (de Arduino). O mundo Arduino camufla itens como esse *.exe para não complicar o desenvolvedor no começo de suas práticas. Mas, é bom saber que esse arquivo *.ino é o código-fonte que você projetará e será carregado na placa ESP32 usando uma plataforma de desenvolvimento chamada Arduino IDE. Estudaremos isso ainda nesse capítulo.

Em C++ todos os erros de sintaxe e de compilação são identificados durante o processo de compilação, antes mesmo do código ser executado. Isso significa que o programa só será executado se você o deixar redondinho em primeiro lugar, sem haver erros de compilação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.3 Resumo das diferenças entre Python e C++

Em Python, tem-se a interpretação linha por linha, ou seja, a execução começa imediatamente após a leitura da linha, o que facilita a depuração durante o desenvolvimento.

Em C++ é compilado em um único passo: o código é transformado em código de máquina antes da execução, resultando em maior desempenho e exigindo a correção de todos os erros antes de poder ser executado.

Essas diferenças influenciam a maneira como você desenvolve e depura programas nas duas linguagens. Python é mais dinâmico e flexível durante o desenvolvimento, enquanto C++ oferece desempenho otimizado e maior controle sobre os recursos do sistema.

4.4 Conceitos básicos

Antes de mergulharmos nos conceitos específicos de C++, é importante revisar alguns princípios fundamentais:

- **Instruções Sequenciais:** são executadas uma após a outra na ordem em que aparecem.
- **Estruturas Condicionais:** permitem a execução de código baseado em condições específicas (if/else).
- **Estruturas de Repetição:** repetem um bloco de código várias vezes (loops).
- **Funções:** blocos de código reutilizáveis que realizam tarefas específicas (`void setup`, `void loop`) e funções personalizadas que serão vistas no curso futuramente.

Explorando um pouco mais cada item apresentado:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Instruções Sequenciais:

O que são? As instruções sequenciais são as operações mais básicas em um programa. Elas são executadas uma após a outra, exatamente na ordem em que são escritas. Pense nelas como uma lista de tarefas que você realiza uma de cada vez.

Exemplo no dia a dia: imagine que você está seguindo uma receita de bolo. As instruções poderiam ser:

1. Misture a farinha e o açúcar.
2. Adicione os ovos.
3. Bata até obter uma massa homogênea.
4. Leve ao forno.

Aqui, você realiza cada passo um após o outro, sem pular ou repetir etapas.

Outro exemplo, mas agora em C++:

```
int a = 5;
int b = 10;
int soma = a + b;
```

Código-fonte 5 - Exemplo básico de lógica de programação
Fonte: Elaborado pelo autor (2024)

Atenção: atente-se ao copiar e colar todos os códigos no Wokwi.com, remova os espaços extras em caixinhas amarelas caso apareçam.

Neste código, as instruções são executadas em sequência:

1. `int a = 5;` define que `a` é igual a 5.
2. `int b = 10;` define que `b` é igual a 10.
3. `int soma = a + b;` calcula a soma de `a` e `b` e armazena o resultado na variável `soma`.

Estruturas Condicionais (If/Else):

Explorando melhor esse fundamento do C++, as estruturas condicionais são essenciais para tomar decisões no código. Elas permitem que diferentes blocos de código sejam executados com base em condições específicas.

Sintaxe:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     if (condição) {
         // código a ser executado se a condição for
verdadeira
     } else {
         // código a ser executado se a condição for falsa
     }
```

Código-fonte 6 - Sintaxe do if/else
Fonte: Elaborado pelo autor (2024)

Exemplo no dia a dia: se você estiver decidindo se deve levar um guarda-chuva ao sair de casa, você pode pensar: “Se estiver chovendo, eu levo o guarda-chuva. Senão, eu não levo”. Agora veja um exemplo usando a temperatura.

Exemplo em C++:

```
     int temperatura = 25;

     if (temperatura > 30) {
         // Executa este código se a temperatura for maior que
30
         Serial.println("Está muito quente!");
     } else {
         // Executa este código se a temperatura não for maior
que 30
         Serial.println("A temperatura está agradável.");
     }
```

Código-fonte 7 - If testando temperatura
Fonte: Elaborado pelo autor (2024)

Neste exemplo, o programa verifica se a temperatura é maior que 30. Se for, ele imprime “Está muito quente!”, caso contrário, imprime “A temperatura está agradável”.

Estruturas de Repetição (Loops):

O que são? As estruturas de repetição permitem que um bloco de código seja executado várias vezes, dependendo de uma condição. Isso é útil quando você precisa realizar a mesma tarefa várias vezes.

Exemplo no dia a dia: imagine que você está enchendo balões para uma festa, e você precisa encher 10 balões. Seria necessário repetir a mesma ação de encher um balão 10 vezes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplo em C++:

```
for (int i = 0; i < 10; i++) {
    Serial.println("Este é o balão número: " +
String(i+1));
}
```

Código-fonte 8 - Exemplo de uso da rotina “for” (para)
Fonte: Elaborado pelo autor (2024)

Neste exemplo, o loop `for` repete a ação de imprimir o número do balão 10 vezes, começando de 0 até 9 (totalizando 10 execuções). “String” é uma função usada para criar ou manipular strings, que são sequências de caracteres. No Arduino, o tipo `String` é uma classe que facilita o trabalho com cadeias de caracteres, oferecendo métodos para concatenar, modificar e analisar strings de maneira mais simples do que com as strings tradicionais em C (`char[]`). No exemplo, String (i+1) converte o número de “i” em um caractere para ser impresso pelo Serial.println.

Outro tipo comum de loop é o `while`, que repete enquanto uma condição for verdadeira:

Sintaxe:

```
while (condição for verdadeira) {
    // código a ser executado se a condição for verdadeira
}
int contador = 0;
while (contador < 10) {
    Serial.println("Contador: " + String(contador));
    contador++;
}
```

Código-fonte 9 - Exemplo de uso do while (enquanto)
Fonte: Elaborado pelo autor (2024)

Aqui, o loop `while` continua repetindo enquanto a variável `contador` for menor que 10.

Funções:

O que são? Funções são blocos de código reutilizáveis que realizam uma tarefa específica. Elas ajudam a organizar o código, evitando a repetição e tornando-o mais fácil de entender e manter.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplo no dia a dia: pense na função como uma receita que você segue sempre que precisa fazer um determinado prato. Você não precisa reescrever a receita toda vez, basta segui-la.

Exemplo em C++: no Arduino e no ESP32, temos duas funções principais que todo programa deve ter: `void setup()` e `void loop()`.

`void setup()`: é onde você coloca o código que deve ser executado apenas uma vez, no início do programa, quando a plaquinha liga e, por isso, só roda uma vez. Aqui, você geralmente inicializa variáveis, configura os pinos de entrada e saída e configura a comunicação serial.

Sintaxe:

```
void setup() {
    // Configurações iniciais
}
```

Código-fonte 10 - Sintaxe do void setup()
Fonte: Elaborado pelo autor (2024)

Exemplo:

```
void setup() {
    Serial.begin(9600); // Inicia a comunicação serial
com 9600 bits por segundo
    pinMode(LED_BUILTIN, OUTPUT); // Configura o LED
construído na placa como saída
}
```

Código-fonte 11 - Exemplo do que pode ser colocado no void setup()
Fonte: Elaborado pelo autor (2024)

A variável LED_BUILTIN indica o pino nativo do LED onboard da placa em uso, geralmente o pino 2 ou 4.

`void loop()`: esta função contém o código que será repetido continuamente enquanto o Arduino/ESP32 estiver ligado, energizado com 3,3V ou 5V. É no void loop() que você coloca a lógica que precisa ser executada repetidamente, como ler sensores e acionar atuadores.

Sintaxe:

```
void loop() {
    // Código repetitivo
}
```

Código-fonte 12 - Sintaxe do void loop()
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplo:

```
void loop() {
        Serial.println("Imprimindo uma frase…");
        delay(1000); // Aguarda 1 segundo. O valor em () está
em milissegundos
    }
```

Código-fonte 13 - Exemplo do que se pode codificar no void loop()
Fonte: Elaborado pelo autor (2024)

Nesse exemplo, a frase “Imprimindo uma frase...” será impressa no terminal a cada 1000 milissegundos ou 1 segundo. O valor declarado no delay( ) é sempre na unidade milissegundos. Então, se você pretende aguardar 1,5s no seu algoritmo, então deve-se usar delay(1500).

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 DECLARAÇÃO DE VARIÁVEIS

Variáveis são utilizadas para armazenar informações que podem ser manipuladas ao longo do programa. As variáveis são posições da memória do microcontrolador. Na família Arduino, existe uma memória do tipo EEPROM que guarda o código-fonte do usuário e permite que você desligue o ESP32 sem perder o código gravado na placa.

Além disso, também existe a memória do tipo RAM no microcontrolador que é onde as variáveis do seu programa ficam armazenadas, e por isso, quando você desligar o microcontrolador, esses dados serão perdidos, mas o programa gravado não.

No C++, as variáveis devem ser declaradas antes de serem usadas, especificando seu tipo.

- **int:** inteiros.
- **float:** números com ponto flutuante.
- **char:** caracteres.
- **boolean:** valores booleanos (true/false).

Exemplo:

```
    int sensorValue = 0;  // Armazena o valor lido de um
sensor
    float temperature = 23.5;  // Armazena a temperatura
    char grade = 'A';  // Armazena uma letra
    boolean isPressed = false;  // Armazena um valor
booleano
```

Código-fonte 14 - Exemplos de declarações de variáveis
Fonte: Elaborado pelo autor (2024)

Atenção: atente-se ao copiar e colar todos os códigos no Wokwi.com, remova os espaços extras em caixinhas amarelas caso apareçam.

Vamos detalhar melhor:

Em C++, todas as variáveis devem ser declaradas antes de serem usadas. Isso significa que você precisa informar ao compilador qual tipo de dado a variável vai

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

armazenar, antes de utilizá-la no seu programa. Isso é diferente de linguagens como Python, onde o tipo de variável é determinado automaticamente.

A estrutura básica para declarar uma variável em C++ é:

```
tipo nomeDaVariavel;
```

Código-fonte 15 - Boas-práticas de como declarar variável
Fonte: Elaborado pelo autor (2024)

Uma boa prática ao declarar variáveis é usar letra minúscula no início e maiúsculas para separar palavras, sem usar espaços, já que é proibido. Por isso utilizamos a escrita “nomeDaVariavel”.

- **tipo:** indica o tipo de dado que a variável vai armazenar.
- **nomeDaVariavel:** você atribui à variável, que deve ser único no escopo onde ela é usada.

5.1 int: Inteiros

Um `int` (abreviação de “integer”) é utilizado para armazenar números inteiros, ou seja, números sem casas decimais.

Exemplo:

```
int idade = 25;
```

Código-fonte 16 - Exemplo de declaração de tipo inteiro
Fonte: Elaborado pelo autor (2024)

- **idade** é uma variável do tipo `int` que armazena o valor 25. Essa variável pode ser usada para armazenar qualquer número inteiro, como -10, 0, ou 1000.

No ESP32, que utiliza a arquitetura de microcontroladores com processador de 32 bits, o tipo `int` em C++ é normalmente um inteiro de 32 bits (4 bytes). Ou seja, ele pode armazenar valores dentro de um intervalo específico.

- **Valor Mínimo:** `-2.147.483.648` (ou `-2^31`).
- **Valor Máximo:** `2.147.483.647` (ou `2^31 - 1`).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esses valores são baseados na capacidade do número de 32 bits de representar valores inteiros, onde um bit é utilizado para o sinal (positivo ou negativo), e os outros 31 bits são usados para o valor numérico.

5.2 float: Números com ponto flutuante

Um `float` é usado para armazenar números com ponto flutuante, ou seja, números que possuem casas decimais.

Exemplo:

```
float temperatura = 36.5;
```

Código-fonte 17 - Exemplo de declaração de tipo float
Fonte: Elaborado pelo autor (2024)

- **temperatura** é uma variável do tipo `float` que armazena o valor 36.5 (36,5 se fosse no padrão brasileiro, mas não podemos usar vírgulas e sim, o ponto decimal para dividir a casa decimal). Essa variável é ideal para representar valores como medidas, porcentagens ou outros números que precisam de precisão decimal.

No ESP32, que utiliza um microcontrolador de 32 bits, o tipo de dado `float` segue a especificação IEEE 754 para números de ponto flutuante de 32 bits (também conhecidos como **single-precision floating point**). Isso permite que o `float` represente uma ampla gama de valores, tanto positivos quanto negativos, com uma certa precisão.

- **Valor Mínimo Positivo Normalizado:** aproximadamente `1.17549 × 10^(-38)`.
- **Valor Máximo:** aproximadamente `3.40282 × 10^(38)`.

Esses valores correspondem ao menor e ao maior valor positivo que podem ser representados por um `float`. Além desses, existem valores especiais como `+0`, `-0`, `infinito` (`inf`), e `NaN` (Not a Number).

Um `float` pode representar aproximadamente 7 dígitos decimais de precisão. Isso significa que, enquanto ele pode lidar com números muito grandes ou muito pequenos, a precisão dos valores pode ser limitada a cerca de 7 dígitos significativos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3 `char`: Caracteres

Um `char` (abreviação de “character”) é usado para armazenar um único caractere, como uma letra, um número ou um símbolo.

Exemplo:

```
char inicial = 'A';
```

Código-fonte 18 - Exemplo de declaração do tipo `char` com uso de aspas simples
Fonte: Elaborado pelo autor (2024)

- **inicial** é uma variável do tipo `char` que armazena o caractere 'A'. Note que os caracteres são sempre colocados entre aspas simples (`' '`).

Se o usuário deseja armazenar vários caracteres em uma única variável em C++, ele não pode usar apenas o tipo `char`, que é destinado a armazenar um único caractere. Em vez disso, ele deve usar uma **string** de caracteres. Em C++, existem duas maneiras principais de armazenar múltiplos caracteres (ou seja, uma cadeia de caracteres) em uma variável:

5.3.1 Array de `char` (C-String)

Um array de `char` pode ser usado para armazenar uma sequência de caracteres. Este é o método tradicional de lidar com strings em C e C++.

Você declara um array de `char` com o tamanho necessário para acomodar todos os caracteres, mais um caractere extra para o **caractere nulo (`\0`)** que marca o fim da string. O caractere nulo (`\0`) é crucial, pois indica o fim da string ao compilador.

Exemplo:

```
char saudacao[15] = "Hey! Como vai?";
```

Código-fonte 19 - Exemplo de declaração do tipo string com uso de aspas duplas
Fonte: Elaborado pelo autor (2024)

- **saudacao** é um array de `char` que pode armazenar até 15 caracteres.
- A string "Hey! Como vai?" ocupa 15 caracteres (H, e, y, !, espaço, C, o, m, o, espaço, v, a, i, ? `\0` no final).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3.2 Acesso aos caracteres

Você pode acessar cada caractere da string individualmente por meio do índice do array.

```
     char letra = saudacao[1];  // letra será 'e', porque todo
vetor começa a contar do [0]
```

Código-fonte 20 - Exemplo de uso de array (vetor)
Fonte: Elaborado pelo autor (2024)

Então teríamos:

[0]: H
[1]: e
[2]: y
[3]: !
[4]:
[5]: C
[6]: o
[7]: m
[8]: o
[9]:
[10]: v
[11]: a
[12]: i
[13]: ?
[14]: /0

Contando de 0 a 14, temos 15 posições.

Algumas observações importantes:

- O tamanho do array é fixo após a declaração. Se você precisar de uma string maior, deve definir um array maior ou usar uma abordagem mais flexível, como `std::string`.
- É necessário se certificar de que a string termina com `\0` para que as funções de manipulação de strings saibam onde a string termina.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3.3 std::string: String da biblioteca padrão do C++

O C++ também oferece a classe `std::string` da biblioteca padrão, que é uma maneira mais conveniente e segura de lidar com strings. Essa abordagem é preferível em muitos casos, especialmente em programas mais complexos.

`std::string` é uma classe que encapsula o comportamento de uma sequência de caracteres, permitindo manipulação de strings de maneira muito mais fácil e intuitiva.

Diferente do array de `char`, o `std::string` pode crescer e encolher dinamicamente, e você não precisa se preocupar em gerenciar manualmente o caractere nulo (`\0`). Ele é inserido automaticamente na última posição.

Exemplo:

```
     #include <string>  //  Inclua  o  cabeçalho  para  usar
std::string

     std::string saudacao = "Hey! Como vai?";
```

Código-fonte 21 - Exemplo de declaração do tipo array sem se preocupar com o seu tamanho
Fonte: Elaborado pelo autor (2024)

- **saudacao** é uma variável do tipo `std::string` que armazena a string "Hey! Como vai?".

Você pode facilmente manipular `std::string` com várias funções e operadores já definidos. Com `std::string`, você pode concatenar strings, acessar caracteres individuais e utilizar várias funções de manipulação de strings com muita facilidade.

```
     saudacao  +=  "  Qual  o  seu  nome?";   //  Concatenar  outra
string apenas usando o operador +=
     char  letra  =  saudacao[28];   //  Acessar  o  vigésimo  nono
caractere, que será 'm'
```

Código-fonte 22 - Exemplo de array se preocupando com o seu tamanho
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Então teríamos:

[0]: H
[1]: e
[2]: y
[3]: !
[4]:
[5]: C
[6]: o
[7]: m
[8]: o
[9]:
[10]: v
[11]: a
[12]: i
[13]: ?
[14]:
[15]: Q
[16]: u
[17]: a
[18]: l
[19]:
[20]: o
[21]:
[22]: s
[23]: e
[24]: u
[25]:
[26]: n
[27]: o
[28]: m
[29]: e
[30]: ?
[31]: /0

Utilizamos o /0 para finalizar a string, isto é, o vetor teria 32 posições, pois saiu do 0 e foi até 31, mas a diferença é que std::string se estica automaticamente quando necessário, ou em outras palavras, sua gestão não é engessada e você não precisa declarar o valor fixo da variável **saudacao**.

Portanto, utilizamos o array de `char` quando precisamos de controle direto e eficiente sobre a memória e as strings são de comprimento fixo; e usamos o std::string quando desejamos facilidade de uso, segurança e funcionalidades avançadas para manipulação de strings sem a preocupação com os espaços da memória. Como o ESP32 possui espaço limitado, é bom trabalhar quando der com o array.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.4 `boolean`: Valores Booleanos

Um `boolean` (ou `bool` em C++) é usado para armazenar um valor de verdade, que pode ser `true` (verdadeiro) ou `false` (falso).

Exemplo:

```
bool estaLigado = true;
```

Código-fonte 23 - Exemplo de declaração do tipo `bool`
Fonte: Elaborado pelo autor (2024)

- **estaLigado** é uma variável do tipo `bool` que armazena o valor `true`. Essa variável pode ser utilizada em condições para tomar decisões no programa, como ligar ou desligar algo.

Cada tipo de dado tem seu próprio propósito, e escolher o tipo certo para cada variável é importante para garantir que o programa funcione corretamente e seja eficiente.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 OPERADORES LÓGICOS

Operadores lógicos são utilizados para combinar ou inverter condições em estruturas condicionais. Os operadores mais comuns incluem:

- **&& (significa AND):** verdadeiro se ambas as condições forem verdadeiras.

  Sintaxe:

```
condição1 && condição2
```

Código-fonte 24 - Exemplo de sintaxe usando operador &&
Fonte: Elaborado pelo autor (2024)

  - Descrição: avalia como `true` (verdadeiro) apenas se ambas as `condição1` e `condição2` forem verdadeiras. Caso contrário, avalia como `false`.

- **|| (significa OR):** verdadeiro se pelo menos uma das condições for verdadeira.

  Sintaxe:

```
condição1 || condição2
```

Código-fonte 25 - Exemplo de sintaxe usando o operador “ou”
Fonte: Elaborado pelo autor (2024)

  - Descrição: avalia como `true` se pelo menos uma das `condição1` ou `condição2` for verdadeira. Avalia como `false` apenas se ambas as condições forem falsas.

- **! (significa NOT):** inverte o valor de uma condição.

  Sintaxe:

```
!condição
```

Código-fonte 26 - Exemplo de sintaxe usando a negação
Fonte: Elaborado pelo autor (2024)

  - Descrição: inverte o valor lógico da `condição`. Se a `condição` for `true`, o operador `!` a torna `false`, e vice-versa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora que você conhece a sintaxe de cada operador lógico, vamos aplicar esses conceitos em exemplos matemáticos.

6.1 Operador AND (`&&`)

Verifique se um número `x` está entre 10 e 20 (inclusive).

```
int x = 15;
if (x >= 10 && x <= 20) {
    Serial.println("O número está entre 10 e 20.");
} else {
    Serial.println("O número não está entre 10 e 20.");
}
```

Código-fonte 27 - Exemplo prático usando o operador E
Fonte: Elaborado pelo autor (2024)

6.2 Operador OR (`||`)

Verifique se um número `y` é menor que 5 ou maior que 15.

```
int y = 18;
if (y < 5 || y > 15) {
    Serial.println("O número é menor que 5 ou maior que
15.");
} else {
    Serial.println("O número está entre 5 e 15.");
}
```

Código-fonte 28 - Exemplo prático usando o operador “ou”
Fonte: Elaborado pelo autor (2024)

6.3 Operador NOT (`!`)

Verifique se um número `z` **não** é igual a 10.

```
int z = 7;
if (!(z == 10)) {
    Serial.println("O número não é igual a 10.");
} else {
    Serial.println("O número é igual a 10.");
}
```

Código-fonte 29 - Exemplo prático usando a negação
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

7 EXERCÍCIOS NÃO AVALIATIVOS

Essa atividade não é avaliativa, caso queira ter um feedback, entre em contato com seu tutor pelo Teams.

7.1 Acendendo um LED

Vamos considerar um exemplo simples onde queremos acender um LED conectado ao ESP32 se um botão for pressionado:

```
     int ledPin = 23;  // pino GPIO 5 conectado ao LED
     int buttonPin = 17;  // pino GPIO 17 conectado ao botão

     void setup() {
         pinMode(ledPin, OUTPUT);  // Define o pino do LED como
saída.
         pinMode(buttonPin, INPUT);  // Define o pino do botão
como entrada
     }

     void loop() {
         int buttonState = digitalRead(buttonPin);  // Lê o
estado do botão

         if (buttonState == HIGH) {
             digitalWrite(ledPin, HIGH);  // Acende o LED
         } else {
             digitalWrite(ledPin, LOW);  // Apaga o LED
         }
     }
```

Código-fonte 30 - Interruptor digital usando botão e LED. Quando o botão é pressionado e mantido pressionado, o LED acende e fica aceso, e quando o botão é liberado, o LED apaga
Fonte: Elaborado pelo autor (2024)

Atenção: atente-se ao copiar e colar todos os códigos no Wokwi.com, remova os espaços extras em caixinhas amarelas caso apareçam.

Explicação do que você está vendo:

Este código em C++ foi escrito para ser executado em uma placa de microcontrolador, como o ESP32 ou Arduino, e tem a função de controlar um LED usando um botão. Vamos dividir o código em partes e explicar cada parte de forma didática:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Declaração de variáveis

```
int ledPin = 23;  // pino GPIO 23 conectado ao LED
int buttonPin = 17;  // pino GPIO 17 conectado ao botão
```

Código-fonte 31 - Declarando os pinos
Fonte: Elaborado pelo autor (2024)

- **int ledPin = 5;** esta linha define uma variável chamada `ledPin` e a associa ao pino GPIO 5 da placa, que está conectado a um LED. Essa variável será usada para controlar o LED.
- **int buttonPin = 17;** aqui, a variável `buttonPin` é definida e associada ao pino GPIO 17, onde um botão está conectado. Essa variável será usada para ler o estado do botão.

Configuração inicial (função `setup()`)

```
void setup() {
    pinMode(ledPin, OUTPUT);  // Define o pino do LED como
saída.
    pinMode(buttonPin, INPUT);  // Define o pino do botão
como entrada
}
```

Código-fonte 32 - Declarando o void setup()
Fonte: Elaborado pelo autor (2024)

- **void setup()** esta função é executada apenas uma vez, quando a placa é ligada ou reiniciada. É usada para configurar os pinos que serão utilizados no programa.
- **pinMode(ledPin, OUTPUT);** aqui, o pino onde o LED está conectado (`ledPin`) é configurado como `OUTPUT` (saída). Isso significa que o pino pode enviar sinais elétricos para acender ou apagar o LED.
- **pinMode(buttonPin, INPUT);** esta linha configura o pino onde o botão está conectado (`buttonPin`) como `INPUT` (entrada). Isso permite que o programa leia se o botão está pressionado ou não.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Loop principal (função `loop()`)

```
     void loop() {
         int buttonState = digitalRead(buttonPin);  // Lê o
estado do botão

         if (buttonState == HIGH) {
             digitalWrite(ledPin, HIGH);  // Acende o LED
         } else {
             digitalWrite(ledPin, LOW);  // Apaga o LED
         }
     }
```

Código-fonte 33 - Declarando o void loop()
Fonte: Elaborado pelo autor (2024)

- **void loop()** esta função é executada repetidamente enquanto a placa estiver ligada. Todo o código dentro do `loop()` será executado em sequência, repetidamente, até que a placa seja desligada.
- **int buttonState = digitalRead(buttonPin);** esta linha lê o estado do pino `buttonPin` (onde o botão está conectado) usando a função `digitalRead()`. O resultado (`HIGH` ou `LOW`) é armazenado na variável `buttonState`.
- **HIGH** significa que o botão está sendo pressionado (ou que há um sinal elétrico alto).
- **LOW** significa que o botão não está sendo pressionado (ou que há um sinal elétrico baixo).
- **if (buttonState == HIGH)** esta condição verifica se o botão está pressionado.

Se `buttonState` for `HIGH`** (o botão está pressionado), o código dentro deste bloco será executado:

```
         digitalWrite(ledPin, HIGH);  // Acende o LED
```

Código-fonte 34 - Função que acende o LED
Fonte: Elaborado pelo autor (2024)

- **digitalWrite(ledPin, HIGH);** envia um sinal elétrico alto para o pino `ledPin`, acendendo o LED.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Se `buttonState` for `LOW`** (o botão não está pressionado), o código dentro do bloco `else` será executado:

```
        digitalWrite(ledPin, LOW);  // Apaga o LED
```

Código-fonte 35 - Função que apaga o LED
Fonte: Elaborado pelo autor (2024)

- **digitalWrite(ledPin, LOW);** envia um sinal elétrico baixo para o pino `ledPin`, apagando o LED.

Portanto, neste exemplo, o LED acenderá quando o botão for pressionado, e apagará quando o botão não estiver pressionado.

7.2 Acendimento do LED usando dois botões simultâneos

Vamos modificar o exemplo do LED e botão anterior para que o LED acenda somente se dois botões forem pressionados simultaneamente.

```
    int ledPin = 23;        // Pino GPIO 23 conectado ao LED
    int buttonPin1 = 16;    // Pino GPIO 16 conectado ao
primeiro botão
    int buttonPin2 = 17;    // Pino GPIO 17 conectado ao
segundo botão

    void setup() {
        pinMode(ledPin, OUTPUT);      // Define o pino do LED
como saída
        pinMode(buttonPin1, INPUT_PULLUP);   // Define o pino
do primeiro botão como entrada
        pinMode(buttonPin2, INPUT_PULLUP);   // Define o pino
do segundo botão como entrada
    }

    void loop() {
        int buttonState1 = digitalRead(buttonPin1);  // Lê o
estado do primeiro botão
        int buttonState2 = digitalRead(buttonPin2);  // Lê o
estado do segundo botão

        if (buttonState1 == LOW && buttonState2 == LOW) {
            digitalWrite(ledPin, HIGH);  // Acende o LED se
ambos os botões estiverem pressionados
        } else {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            digitalWrite(ledPin, LOW);    // Apaga o LED se
pelo menos um dos botões não estiver pressionado
        }
    }
```

Código-fonte 36 - Exemplo usando dois botões que ao serem pressionados juntos, o LED acende e se mantém aceso; se soltar um botão, o LED apaga
Fonte: Elaborado pelo autor (2024)

Configure os botões para serem usados via teclado. Para isso, clique no botão do seu circuito com a simulação parada e associe o botão com uma tecla do seu teclado. Assim, terá condições de pressionar 2 botões ao mesmo tempo.

Na linha **buttonState1 == HIGH && buttonState2 == HIGH** estamos usando o operador lógico `&&` (AND) para verificar se os dois botões estão pressionados simultaneamente. Ambos os estados (`buttonState1` e `buttonState2`) precisam ser `HIGH` para que a condição seja verdadeira.

- **digitalWrite(ledPin, HIGH):** se a condição for verdadeira, o LED acenderá.
- **digitalWrite(ledPin, LOW):** se a condição não for atendida (ou seja, se um ou ambos os botões não estiverem pressionados), o LED permanecerá apagado.

7.3 Verificando se é par ou ímpar

Verifique se um número é par ou ímpar e acenda um LED para indicar o resultado. O usuário fornece um número inteiro na variável **número**. Se o número for par, o LED acende; se for ímpar, o LED permanece apagado.

```
    int ledPin = 5;  // Pino GPIO 5 conectado ao LED
    int numero = 7;  // Número fornecido pelo usuário

    void setup() {
        pinMode(ledPin, OUTPUT);  // Define o pino do LED como
saída

        if (numero % 2 == 0) {
            digitalWrite(ledPin, HIGH);  // Acende o LED se o
número for par
        } else {
            digitalWrite(ledPin, LOW);   // Apaga o LED se o
número for ímpar
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        }
    }

    void loop() {
        // Nada a fazer aqui, pois o estado do LED é decidido
no setup()
    }
```

Código-fonte 37 - Acende um LED se o número declarado for par
Fonte: Elaborado pelo autor (2024)

Note que todo o código foi posto na função void setup() { } porque o número está fixo na variável “número” e, por isso, não faria sentido testar mais de uma vez. Neste caso, o teste é feito apenas uma vez quando você inicia o chip ESP32.

- **if (numero % 2 == 0)** verifica se o número é divisível por 2. Se o resto da divisão for 0, o número é par.
- **digitalWrite(ledPin, HIGH/LOW)** liga ou desliga o LED baseado no resultado da verificação.

7.4 Comparando números

Nesse exemplo, vamos comparar dois números e acender um LED se eles forem iguais. O usuário fornece dois números inteiros em **num1** e **num2**. Se os dois números forem iguais, o LED acende; caso contrário, o LED permanece apagado.

```
    int ledPin = 23;  // Pino GPIO 23 conectado ao LED
    int num1 = 5;    // Primeiro número fornecido pelo usuário
    int num2 = 5;    // Segundo número fornecido pelo usuário

    void setup() {
        pinMode(ledPin, OUTPUT);  // Define o pino do LED como
saída

        if (num1 == num2) {
            digitalWrite(ledPin, HIGH);  // Acende o LED se
os números forem iguais
        } else {
            digitalWrite(ledPin, LOW);   // Apaga o LED se os
números forem diferentes
        }
    }

    void loop() {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        // Nada a fazer aqui, pois o estado do LED é decidido
no setup()
    }
```

Código-fonte 38 - Acende um LED se 2 números forem iguais
Fonte: Elaborado pelo autor (2024)

Note mais uma vez que todo o código foi posto na função void setup() { } porque os números estão fixos nas variáveis “num1” e “num2” e, por isso, não faria sentido testar mais de uma vez. Nesse caso, o teste é feito apenas uma vez quando você inicia o chip ESP32.

- **if (num1 == num2)** compara os dois números fornecidos pelo usuário. Se eles forem iguais, a condição é verdadeira.
- **digitalWrite(ledPin, HIGH/LOW)** liga o LED se os números forem iguais e o apaga se forem diferentes.

7.5 Evoluindo o teste de números

Determine se um número é positivo, negativo ou zero, e acenda um LED específico para cada caso. O usuário fornece um número inteiro. O LED1 acende se o número for positivo. O LED2 acende se o número for negativo. O LED3 acende se o número for zero.

```
    int ledPinPositivo = 23;  // Pino GPIO 23 conectado ao
LED para números positivos
    int ledPinNegativo = 22;  // Pino GPIO 22 conectado ao
LED para números negativos
    int ledPinZero = 16;      // Pino GPIO 16 conectado ao
LED para zero
    int numero = 0;         // Número fornecido pelo usuário

    void setup() {
        pinMode(ledPinPositivo, OUTPUT);  // Define o pino
para número positivo como saída
        pinMode(ledPinNegativo, OUTPUT);  // Define o pino
para número negativo como saída
        pinMode(ledPinZero, OUTPUT);      // Define o pino
para zero como saída
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        if (numero > 0) {
            digitalWrite(ledPinPositivo, HIGH);  // Acende o
LED para número positivo
        } else if (numero < 0) {
            digitalWrite(ledPinNegativo, HIGH);  // Acende o
LED para número negativo
        } else {
            digitalWrite(ledPinZero, HIGH);      // Acende o
LED para zero
        }
    }

    void loop() {
        // Nada a fazer aqui, pois o estado do LED é decidido
no setup()
    }
```

Código-fonte 39 - Testando novamente números, mas agora positivos e negativos
Fonte: Elaborado pelo autor (2024)

- **if (numero > 0)** verifica se o número é positivo.
- **else if (numero < 0)** verifica se o número é negativo.
- **else** se nenhuma das condições anteriores for verdadeira, o número deve ser zero.

Cada condição acende um LED específico para indicar o estado do número (positivo, negativo ou zero).

7.6 Evoluindo a multiplicidade

Verifique se um número é múltiplo de 3 e/ou 5 e acenda um LED correspondente. O usuário fornece um número inteiro. O LED1 acende se o número for múltiplo de 3. O LED2 acende se o número for múltiplo de 5. Se o número for múltiplo de ambos (3 e 5), ambos os LEDs acendem.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     int ledPin = 23;   // Pino GPIO 23 conectado ao LED para
múltiplos de 3
     int ledPin = 22;   // Pino GPIO 22 conectado ao LED para
múltiplos de 5
     int numero = 15;   // Número fornecido pelo usuário

     void setup() {
         pinMode(ledPin,  OUTPUT);   //  Define  o  pino  para
múltiplos de 3 como saída
         pinMode(ledPin,  OUTPUT);   //  Define  o  pino  para
múltiplos de 5 como saída

         if (numero % 3 == 0) {
             digitalWrite(ledPin, HIGH);  // Acende o LED se o
número for múltiplo de 3
         }

         if (numero % 5 == 0) {
             digitalWrite(ledPin, HIGH);  // Acende o LED se o
número for múltiplo de 5
         }
     }

     void loop() {
         // Nada a fazer aqui, pois o estado dos LEDs é decidido
no setup()
     }
```

Código-fonte 40 - Outro exemplo de teste de números
Fonte: Elaborado pelo autor (2024)

- **numero % 3 == 0** verifica se o número é divisível por 3 (ou seja, se o resto da divisão por 3 é zero).
- **numero % 5 == 0** verifica se o número é divisível por 5.

Dependendo do resultado, um LED específico acende. Se o número for múltiplo de ambos (por exemplo, 15), ambos os LEDs acenderão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

CAMPUS Component Pvt. Ltd. What is the ESP8266 Wi-Fi Module?. **Medium**, 25 may 2023. Disponível em: <https://campuscomponent11.medium.com/what-is-the-esp8266-wi-fi-module-ef8f6013ec03>. Acesso em: 4 set. 2024.

ESP32 Modules and Boards. **Espressif**. 2020. Disponível em: https://docs.espressif.com/projects/esp-idf/en/v4.2/esp32/hw-reference/modules-and-boards.html. Acesso em: 6 set. 2024.

GAURAV, Kumar. What is ESP 8266/ESP 32. **Techeonics**, 12 july 2022. Disponível em: <https://techeonics.com/what-are-esp-microcontrollers/>. Acesso em: 4 set. 2024.

LOUIS, Leo. Working principle of Arduino and using it. **International Journal of Control, Automation, Communication and Systems (IJCACS)**, v. 1, n. 2, p. 21-29, 2016.

MCROBERTS, Michael. **Arduino Básico-2ª edição: Tudo sobre o popular microcontrolador Arduino**. Novatec Editora, 2015.

MURAD, S. A. Zainol *et al*. Design of aquaponics water monitoring system using Arduino microcontroller. In: AIP Conference Proceedings. AIP Publishing, 2017. CORDOVA, H.; TORT, A. C. Medida de g com a placa Arduino em um experimento simples de queda livre. **Revista Brasileira de Ensino de Física**, v. 38, n. 2, p. e2308, 2016.

SIHOMBING, P. *et al*. Automated hydroponics nutrition plants systems using arduino uno microcontroller based on android. In: **Journal of Physics: Conference Series**. IOP Publishing, 2018. p. 012014.

SRIVASTAVA, Prakhar; BAJAJ, Mohit; RANA, Ankur Singh. Overview of ESP8266 Wi-Fi module based smart irrigation system using IOT. In: 2018 Fourth International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics (AEEICB). **IEEE**, 2018. p. 1-5.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com